S20 · EVIDENCIA DE APRENDIZAJE 2

# Sistema de información del parqueadero “Autos Colombia”

Diseño de Sistema de Información – Iteración 2: Gestión de usuarios y gestión de celdas

| | |
|---|---|
| Estudiante | Cristian David España Figueroa |
| Programa | Tecnología en Desarrollo de Software |
| Institución | Institución Universitaria Digital de Antioquia (IU Digital) |
| Asignatura | Ingeniería de Software |
| Docente | Esteban Montoya Molina |
| Código curso | PREICA2602B020081 |
| Periodo | 2026_2_B2 |
| Repositorio GitHub | https://github.com/davide1828/Autos-Colombia.git |
| Diseño en Figma | https://www.figma.com/design/DhCeukFFGQef4egiIl3UbY/ |
| Ciudad y fecha | Medellín, Colombia · Noviembre de 2026 |

1. Introducción y Enfoque de la Iteración 2
2. Requerimientos Funcionales y No Funcionales (Gestión de Usuarios y Celdas)
3. Diagramas de Casos de Uso en STARUML
4. Historias de Usuario Asociadas (Formato Ágil y Gherkin)
5. Mockups de Interfaz de Usuario de las Historias de Gestión
6. Diagramas de Secuencia de las Iteraciones 1 y 2 en STARUML
7. Diagrama de Componentes de las Iteraciones 1 y 2 en STARUML
8. Formato de Plan de Pruebas Resumido (Gestión de Usuarios y Celdas)
9. Repositorio en GitHub y Evidencias de Código Fuente
10. Matriz de Trazabilidad entre Representaciones
11. Conclusiones

# 1. Introducción y Enfoque de la Iteración 2

El sistema de información del parqueadero **“Autos Colombia”** es una solución integral diseñada bajo principios de ingeniería de software para gestionar las operaciones cotidianas de un parqueadero que presta servicios de mensualidad. Tras haber completado y validado en la **Iteración 1** el núcleo operativo de patio (registro de ingreso, salida, permanencia, control de bahías y novedades bajo una arquitectura limpia y desacoplada), la presente **Iteración 2** aborda la gobernabilidad, administración y seguridad del sistema.

En esta iteración se incorporan dos ejes fundamentales: la **Gestión de Usuarios con control de acceso basado en roles (RBAC)** y la **Gestión Integral de Celdas**, respaldadas por un subsistema de **Auditoría Inmutable** a nivel de persistencia relacional. El sistema evoluciona de un esquema monousuario a una plataforma multiusuario donde se diferencian estrictamente las atribuciones del *Administrador* (gestión de identidades, configuración del patio, asignación de celdas y auditoría) y del *Encargado* (operación diaria y actualización de su propia clave).

Todo el diseño formal presentado en este documento se encuentra respaldado por su implementación ejecutable en el backend (Go 1.22 con arquitectura hexagonal), pruebas automatizadas con 0 fallos, base de datos relacional PostgreSQL 16 con disparadores de integridad transaccional, y artefactos UML modelados en **StarUML**.

Un requerimiento funcional define los servicios y comportamientos del software en respuesta a estímulos específicos. Para la Iteración 2, los requerimientos se formulan alineados con las reglas de negocio implementadas y garantizadas tanto en la capa de dominio como mediante restricciones y triggers en la base de datos.

• **RN-07 (Autenticación y Sesión Segura):** Todo acceso al sistema exige credenciales individuales. Las contraseñas se almacenan mediante hash con sal criptográfica individual (PBKDF2-SHA256, 100.000 iteraciones). Se bloquean solicitudes tras 5 intentos fallidos consecutivos por IP.
• **RN-08 (Gobernabilidad y Último Administrador):** El sistema debe asegurar en todo momento la presencia de al menos un usuario activo con rol ADMINISTRADOR. Ninguna operación puede degradar o inactivar al último administrador existente.
• **RN-09 (Integridad de Celdas Ocupadas):** Una celda en estado OCUPADA no puede pasar a INACTIVA ni ser modificada en su código o tipo mientras mantenga un vehículo dentro.
• **RN-10 (No Solapamiento de Mensualidades):** No se permiten periodos de mensualidad superpuestos para el mismo cliente.
• **RN-11 (Auditoría Inmutable):** Toda modificación administrativa sobre usuarios, celdas y clientes queda registrada en la tabla de auditoría, la cual rechaza cualquier UPDATE o DELETE mediante trigger transaccional.

## Requerimientos Funcionales (RF-08 a RF-21)

| Id | Requerimiento Funcional | Regla | Prioridad |
|---|---|---|---|
| RF-08 | El sistema debe permitir a los usuarios (Administrador y Encargado) autenticarse mediante login y contraseña, retornando un token de sesión seguro (Bearer) con expiración y el perfil del usuario. | RN-07 | Alta |
| RF-09 | El sistema debe controlar los accesos según el rol (RBAC): restringir la gestión de usuarios, celdas, vehículos y auditoría al ADMINISTRADOR, y permitir al ENCARGADO operar patio y cambiar su clave. | RN-07 | Alta |
| RF-10 | El sistema debe permitir al Administrador registrar nuevos usuarios indicando login único (3-30 caracteres alfanuméricos), nombre, rol (ADMINISTRADOR/ENCARGADO) y contraseña inicial segura. | RN-07 | Alta |
| RF-11 | El sistema debe permitir al Administrador actualizar datos de usuarios y alternar su estado (activo/inactivo), bloqueando la inactivación o degradación del último administrador activo. | RN-08 | Alta |
| RF-12 | El sistema debe permitir al usuario autenticado actualizar su contraseña personal validando su contraseña actual y verificando que la nueva clave cumpla con los estándares de robustez. | RN-07 | Media |
| RF-13 | El sistema debe permitir al Administrador restablecer la contraseña de cualquier usuario sin necesidad de conocer la clave anterior, aplicando las directivas de seguridad. | RN-07 | Media |
| RF-14 | El sistema debe revocar de inmediato el acceso de cualquier usuario que sea inactivado o degradado de rol, consultando la base de datos en cada petición protegida. | RN-07, RN-08 | Alta |
| RF-15 | El sistema debe permitir al Administrador crear celdas con código alfanumérico único en mayúsculas de hasta 10 caracteres (ej. A-01) y tipo de vehículo (CARRO/MOTO) en estado inicial DISPONIBLE. | RN-03 | Alta |
| RF-16 | El sistema debe permitir al Administrador alternar el estado de una celda entre DISPONIBLE e INACTIVA (por mantenimiento o daño), impidiendo inhabilitarla si se encuentra ocupada. | RN-09 | Alta |

| Id | Requerimiento Funcional | Regla | Prioridad |
|---|---|---|---|
| RF-17 | El sistema debe permitir al Administrador editar código o tipo de una celda, validando que no tenga ningún vehículo dentro al momento de la modificación. | RN-03, RN-09 | Media |
| RF-18 | El sistema debe permitir registrar clientes con su documento, datos de contacto, vehículos y registrar mensualidades validando que no existan periodos solapados para el mismo cliente. | RN-01, RN-10 | Alta |
| RF-19 | El sistema debe permitir al Administrador corregir el tipo de un vehículo (CARRO/MOTO), validando que no se encuentre dentro del patio para no violar la compatibilidad con su celda. | RN-03 | Media |
| RF-20 | El sistema debe registrar automáticamente en la bitácora de auditoría cada creación, modificación o inactivación de usuarios, celdas y vehículos, impidiendo su edición o borrado. | RN-11 | Alta |
| RF-21 | El sistema debe permitir al Administrador consultar el histórico cronológico de auditoría con datos de momento UTC, usuario responsable, entidad afectada, ID y detalle. | RN-11 | Media |

## Requerimientos No Funcionales (RNF-08 a RNF-13)

| Id | Categoría | Requerimiento No Funcional Verificable | Métrica / Criterio |
|---|---|---|---|
| RNF-08 | Seguridad Criptográfica | Las contraseñas deben almacenarse hasheadas con PBKDF2 (HMAC-SHA256, sal de 16 bytes, 100.000 iteraciones). Las validaciones deben ejecutarse en tiempo constante para mitigar ataques de temporización. | 100 % pruebas unitarias superadas en seguridad; uso de subtle.ConstantTimeCompare. |
| RNF-09 | Anti Fuerza Bruta | El endpoint de login debe limitar los intentos fallidos por IP (máximo 5 fallos por minuto). Si se excede, debe responder HTTP 429 con cabecera Retry-After: 60. | Validado en pruebas de HTTP API con RateLimiter por IP. |
| RNF-10 | Integridad Concurrente | Las operaciones críticas sobre celdas, vehículos y validación del último administrador deben ejecutarse en transacciones ACID con bloqueos pesimistas (FOR SHARE / FOR UPDATE). | Sin condiciones de carrera bajo pruebas concurrentes en PostgreSQL. |
| RNF-11 | Inviolabilidad | La tabla de auditoría debe ser inmutable a nivel de base de datos. Ningún usuario o proceso puede modificar ni borrar registros de bitácora. | Trigger trg_auditoria_inmutable que arroja excepción ante UPDATE o DELETE. |
| RNF-12 | Rendimiento Auth | La verificación de autenticación y extracción del actor en cada petición no debe incrementar la latencia en más de 5 ms. | Medido en peticiones HTTP con servidor en Go. |
| RNF-13 | Mantenibilidad | Arquitectura hexagonal en Go: desacoplamiento absoluto de dominio, casos de uso, HTTP y persistencia, con doble adaptador (PostgreSQL y Memoria). | Casos de uso ejecutables sin base de datos externa para pruebas ágiles. |

El diagrama de casos de uso modela el comportamiento del sistema desde el punto de vista de los actores externos, capturando las metas funcionales del negocio. El modelo fue diseñado y exportado en **StarUML** (archivo **docs/diagramas/parqueadero_iteracion_2.mdj**), estructurando dos actores y seis casos de uso principales.

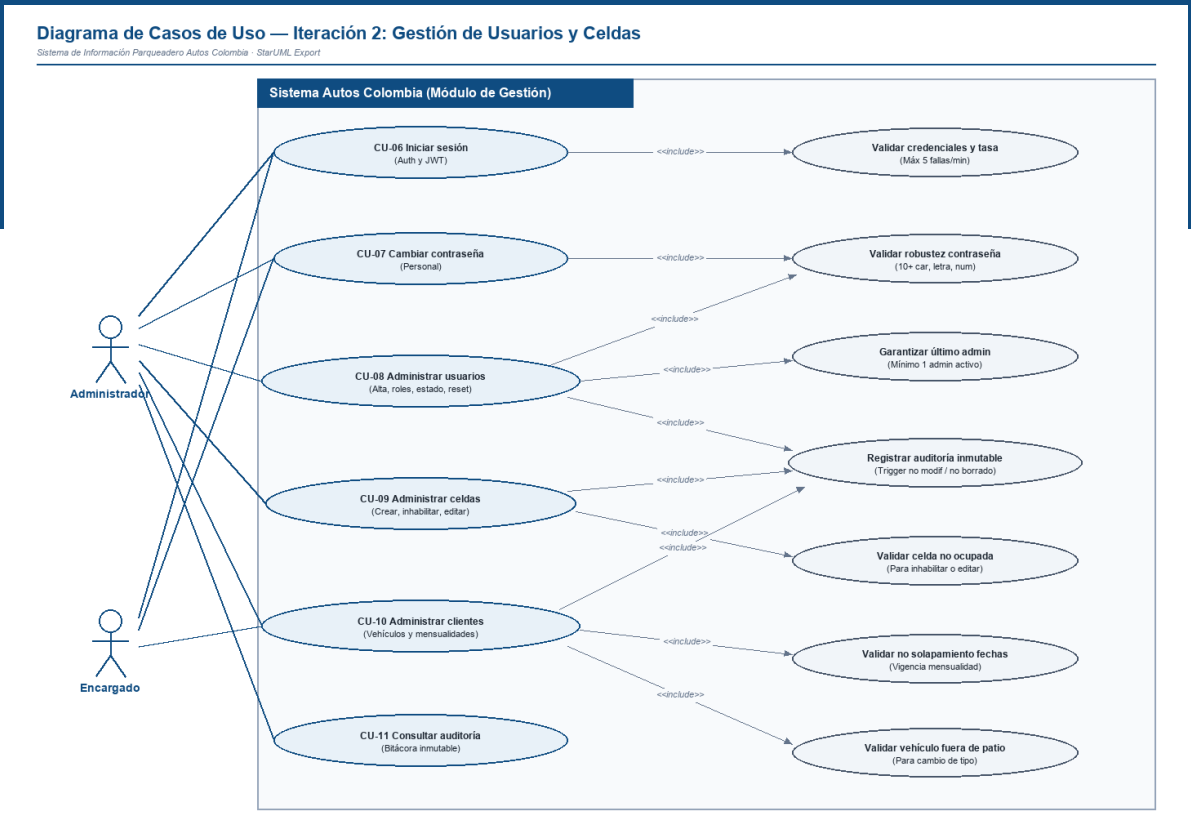



*Figura 1. Diagrama de Casos de Uso de la Iteración 2 (StarUML / docs/diagramas/08_casos_uso_iteracion_2.puml)*

## Especificación de los Casos de Uso

| Caso de Uso | Actores | Descripción | Precondición | Postcondición | Inclusiones <> |
|---|---|---|---|---|---|
| CU-06 Iniciar sesión | Admin, Encargado | Autentica al usuario en el sistema mediante usuario y clave, generando token de sesión. | Usuario registrado y activo. | Token Bearer emitido; reinicio de intentos fallidos. | Validar credenciales y tasa. |
| CU-07 Cambiar contraseña | Admin, Encargado | Permite actualizar la contraseña personal confirmando la clave actual. | Sesión activa. | Contraseña actualizada; auditoría registrada. | Validar robustez contraseña. |
| CU-08 Administrar usuarios | Administrador | Permite crear usuarios, editar nombres, alternar roles, inactivar y resetear claves. | Admin autenticado. | Usuario guardado; bitácora registrada. | Validar robustez clave. Garantizar último admin. Registrar auditoría inmutable. |

| Caso de Uso | Actores | Descripción | Precondición | Postcondición | Inclusiones <> |
|---|---|---|---|---|---|
| CU-09 Administrar celdas | Administrador | Permite registrar celdas, editar código/tipo e inhabilitar por mantenimiento. | Admin autenticado. | Celda creada/actualizada; auditoría registrada. | Validar celda no ocupada. Registrar auditoría inmutable. |
| CU-10 Administrar clientes | Admin, Encargado | Registra clientes, asocia vehículos y renueva mensualidades sin solape. | Usuario autenticado / Admin. | Contrato registrado; vigencia actualizada. | Validar no solapamiento fechas. Validar vehículo fuera de patio. Registrar auditoría inmutable. |
| CU-11 Consultar auditoría | Administrador | Consulta cronológica descendente de la bitácora inalterable de auditoría. | Admin autenticado. | Registros devueltos sin modificación. | Ninguna (consulta pura). |

Cada historia de usuario sigue el formato ágil estándar *Como [rol], quiero [capacidad], para [beneficio]*, acompañada de criterios de aceptación estructurados en lenguaje Gherkin (*Dado / Cuando / Entonces*).

**Como** Administrador del parqueadero, **quiero** registrar, modificar y administrar las cuentas de los usuarios del sistema asignando roles (ADMINISTRADOR / ENCARGADO), **para** controlar el acceso a la plataforma y aplicar el principio de menor privilegio.

• **CA-06.1 Registro válido:** Dado que el Administrador diligencia un login único (3-30 car), nombre válido, clave robusta y rol, Cuando solicita la creación, Entonces el sistema crea el usuario activo, almacena el hash PBKDF2, registra la auditoría y retorna HTTP 201 Created.
• **CA-06.2 Login duplicado:** Dado que ya existe un usuario con el mismo login, Cuando se intenta registrar, Entonces el sistema rechaza con HTTP 409 Conflict ('ya existe un usuario con ese nombre de usuario').
• **CA-06.3 Contraseña débil:** Dado que se ingresa una clave menor a 10 caracteres o sin números/letras, Cuando se envía la solicitud, Entonces responde HTTP 400 Bad Request explicando la política.
• **CA-06.4 Inactivación de usuario:** Dado un Encargado activo, Cuando el Admin cambia su estado a inactivo, Entonces el sistema persiste el cambio y cualquier uso posterior de su token es denegado con HTTP 401.
• **CA-06.5 Protección del último administrador activo:** Dado un único administrador activo, Cuando se intenta inactivar o cambiar su rol, Entonces el sistema rechaza con HTTP 409 Conflict ('debe quedar al menos un administrador activo').

## HU-07 · Autenticación y cambio de credenciales (Relacionada con: RF-08, RF-12, RF-13)

**Como** usuario del sistema (Administrador o Encargado), **quiero** iniciar sesión de forma segura y actualizar mi contraseña personal, **para** acceder a las funciones operativas y mantener mi cuenta protegida.

• **CA-07.1 Login exitoso:** Dado un usuario activo y su contraseña correcta, Cuando envía credenciales a /api/login, Entonces recibe HTTP 200 OK con token Bearer, expiración y perfil.
• **CA-07.2 Credenciales erróneas:** Dado un usuario o contraseña incorrectos, Cuando solicita login, Entonces recibe HTTP 401 Unauthorized ('usuario o contraseña incorrectos') y suma 1 fallo a la IP.
• **CA-07.3 Bloqueo por fuerza bruta:** Dado que una IP acumula 5 intentos fallidos en < 60s, Cuando reintenta, Entonces recibe HTTP 429 Too Many Requests con cabecera Retry-After: 60.
• **CA-07.4 Cambio de contraseña personal:** Dado un usuario autenticado que digita su clave actual correcta y una nueva clave válida, Cuando confirma el cambio, Entonces actualiza el hash PBKDF2 y retorna HTTP 200 OK.
• **CA-07.5 Confirmación de clave actual errónea:** Dado que el usuario falla al digitar su clave actual, Cuando intenta cambiarla, Entonces el sistema responde HTTP 400 Bad Request sin alterar la clave.

## HU-08 · Gestión y mantenimiento de celdas (Relacionada con: RF-15, RF-16, RF-17)

**Como** Administrador, **quiero** registrar celdas y habilitar o inhabilitar bahías por obras o mantenimiento, **para** reflejar la disponibilidad real del patio y evitar asignaciones erróneas.

• **CA-08.1 Registro de celda:** Dado un código único (ej. A-05) y tipo CARRO/MOTO, Cuando se registra, Entonces se guarda en estado DISPONIBLE y retorna HTTP 201 Created.
• **CA-08.2 Inhabilitación por mantenimiento:** Dado una celda DISPONIBLE, Cuando el Admin cambia su estado a INACTIVA, Entonces el sistema la excluye de los selectores operativos de ingreso.

• **CA-08.3 Bloqueo en celda ocupada:** Dado una celda en estado OCUPADA con un vehículo dentro, Cuando el Admin intenta inhabilitarla o cambiar su tipo, Entonces el sistema rechaza con HTTP 409 Conflict ('la celda está ocupada; registre primero la salida del vehículo').

**Como** operador del parqueadero, **quiero** registrar clientes, asociar sus vehículos y renovar contratos de mensualidad, **para** autorizar el acceso exclusivo a vehículos con pago al día.

• **CA-09.1 Mensualidad válida:** Dado un cliente y un rango de fechas sin conflicto, Cuando se agrega, Entonces se genera la mensualidad y se actualiza la vigencia del cliente.
• **CA-09.2 Rechazo por solapamiento:** Dado un cliente con mensualidad vigente, Cuando se intenta registrar un periodo que se cruza con el existente, Entonces el sistema rechaza con HTTP 409 Conflict.
• **CA-09.3 Modificación de tipo con vehículo dentro:** Dado un vehículo alojado en patio, Cuando se intenta modificar su tipo, Entonces se bloquea con HTTP 409 Conflict para mantener la compatibilidad con la celda.

## HU-10 · Auditoría inmutable de operaciones (Relacionada con: RF-20, RF-21)

**Como** Administrador, **quiero** consultar la bitácora inalterable de auditoría, **para** mantener la trazabilidad de las operaciones y responder ante reclamaciones.

• **CA-10.1 Registro automático:** Toda modificación administrativa inserta una fila en auditoria con marca de tiempo UTC y actor.
• **CA-10.2 Inmutabilidad por trigger:** Cualquier sentencia UPDATE o DELETE sobre auditoria es abortada por la base de datos con excepción irrevocable.

Los mockups fueron diseñados manteniendo la línea visual corporativa del parqueadero Autos Colombia: paleta sobria (Azul #0F4C81, Asfalto #1C2530, fondo #F8FAFC), tipografía clara y jerarquizada, tarjetas de contenido, badges de estado con alto contraste y formularios modales accesibles.

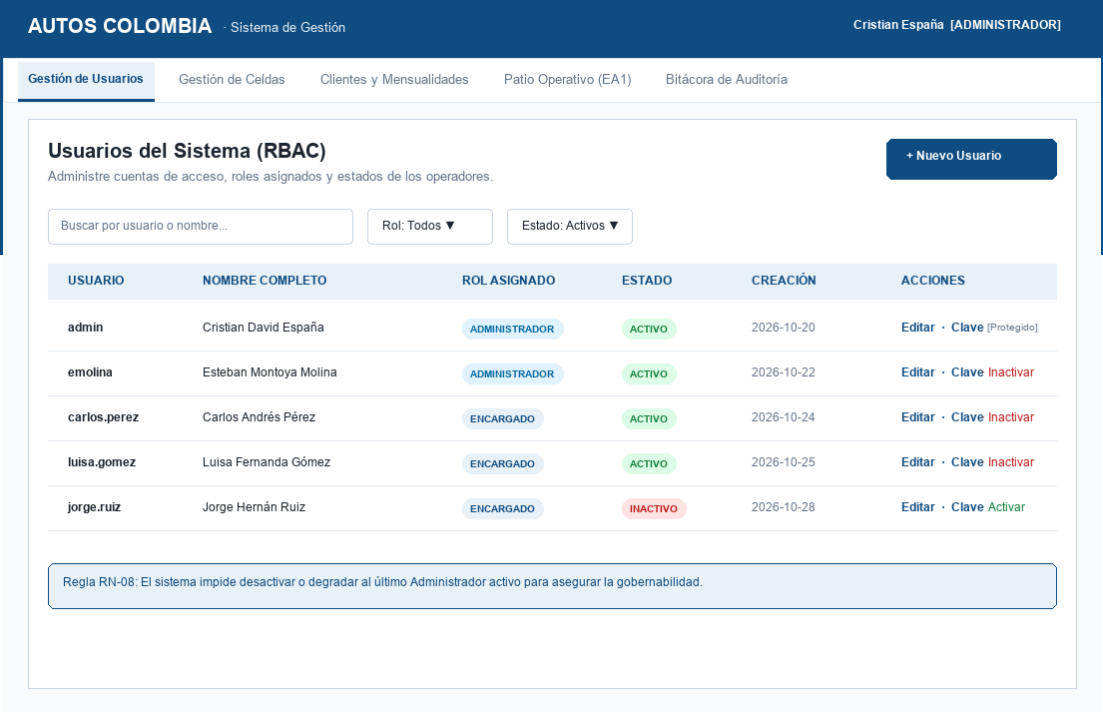



*Figura 2. Mockup de Gestión de Usuarios del Sistema (Listado RBAC, estados, filtros y protección del último administrador)*

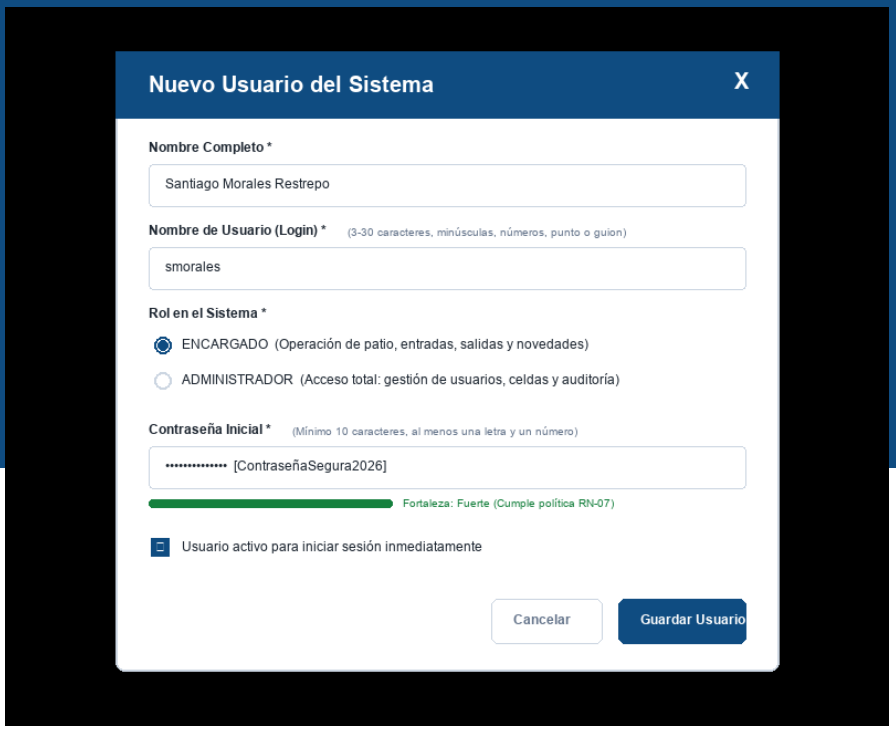



*Figura 3. Mockup de Creación y Edición de Usuario (Formulario modal con indicador de fortaleza de contraseña RN-07 y asignación de rol)*

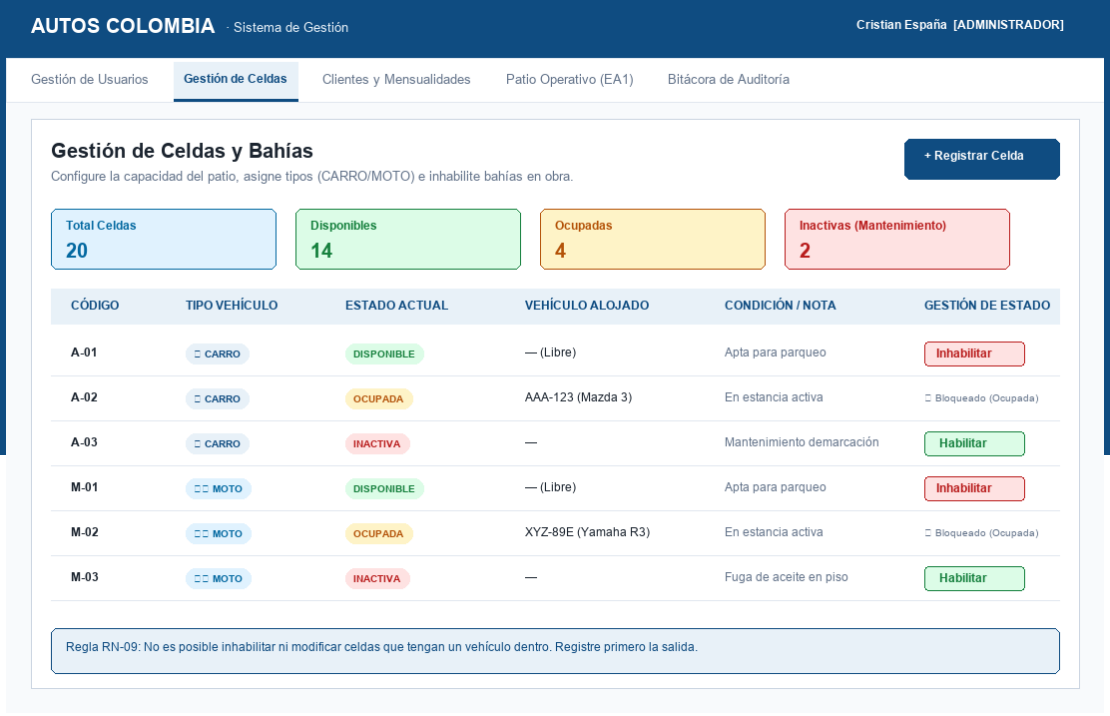



*Figura 4. Mockup de Gestión de Celdas y Bahías (Resumen de capacidad, tipos CARRO/MOTO y conmutador de inhabilitación por mantenimiento RN-09)*

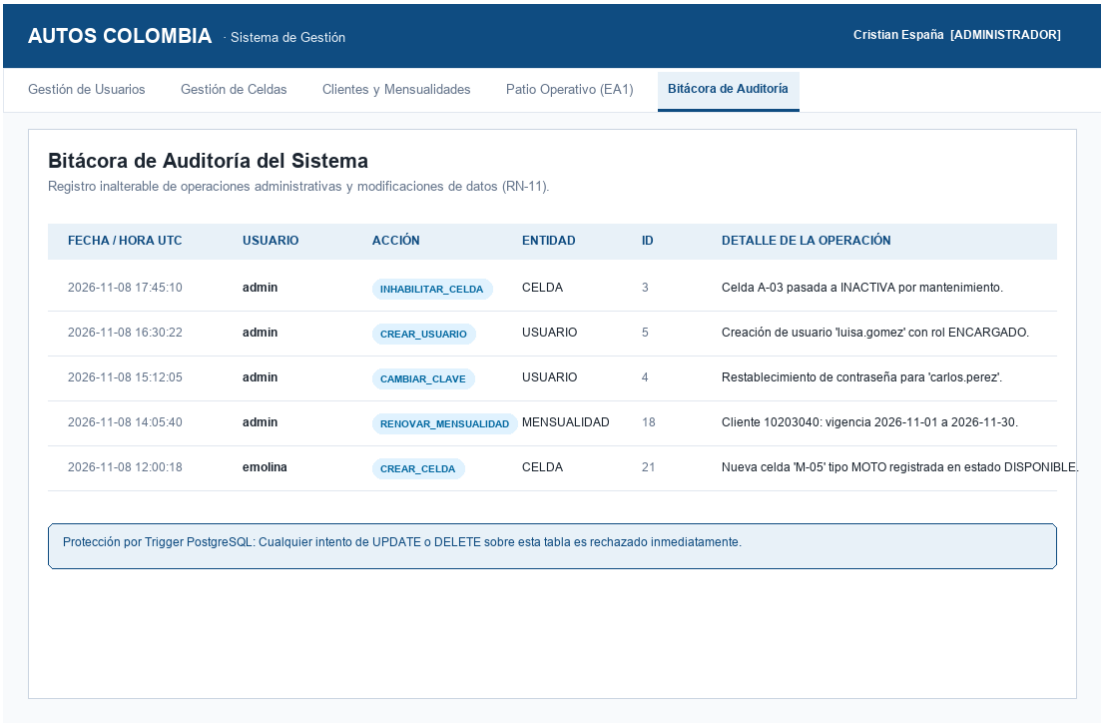



*Figura 5. Mockup de Bitácora de Auditoría Inmutable (Consulta cronológica de eventos administrativos protegida por trigger RN-11)*

Los diagramas de secuencia modelan la interacción temporal y el intercambio de mensajes entre los componentes del sistema (Frontend, Capa HTTP, Servicios de Aplicación, Reglas de Dominio y Base de Datos). Se incluyen los diagramas representativos tanto de la **Iteración 1** (ingreso y salida) como de la **Iteración 2** (login con PBKDF2 y gestión de celdas con auditoría).

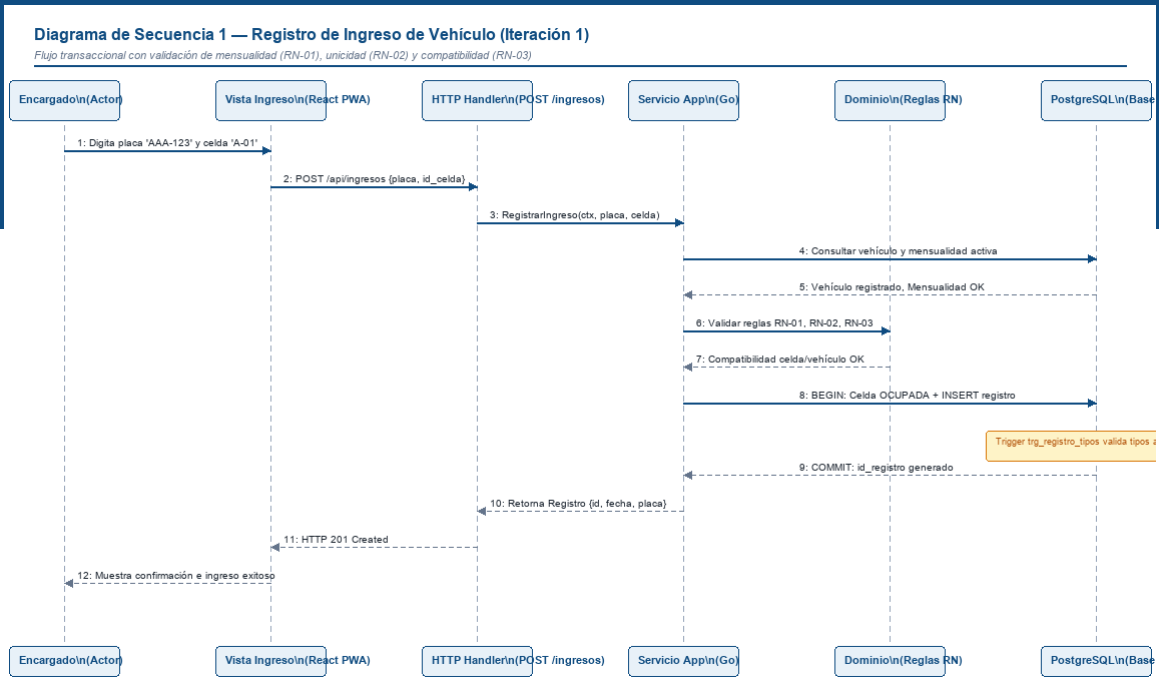



*Figura 6. Diagrama de Secuencia 1 — Registro de Ingreso de Vehículo (Iteración 1: validaciones de mensualidad, celda disponible y trigger de tipos)*

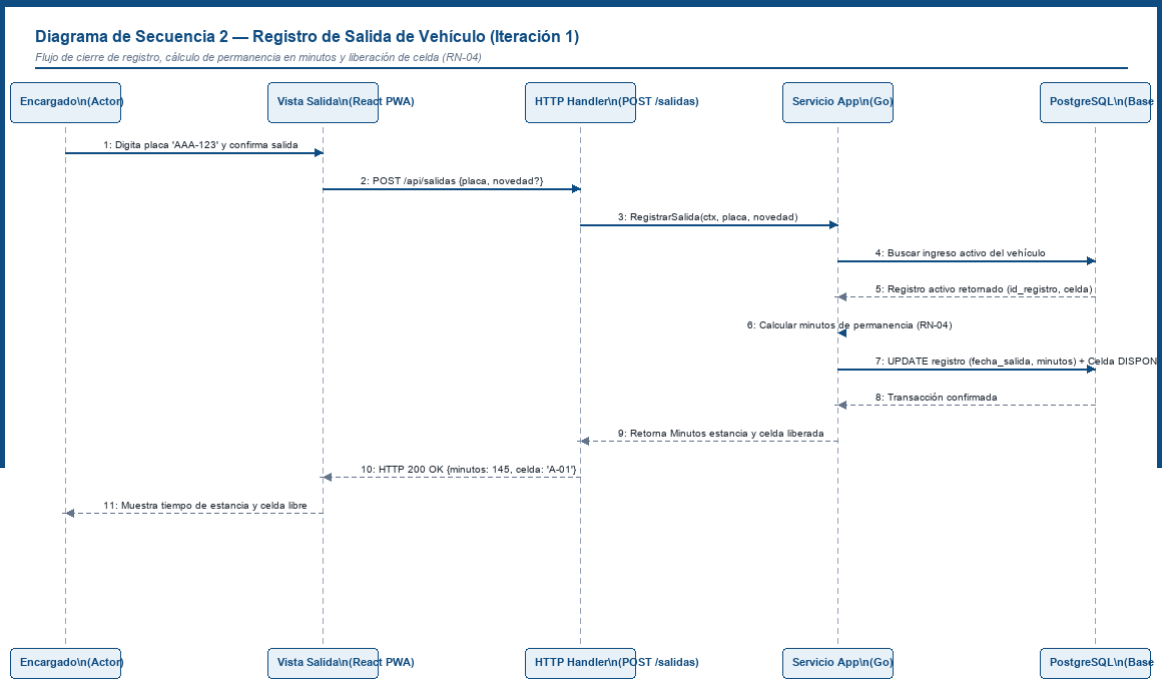



*Figura 7. Diagrama de Secuencia 2 — Registro de Salida de Vehículo (Iteración 1: cálculo de permanencia y liberación de celda RN-04)*

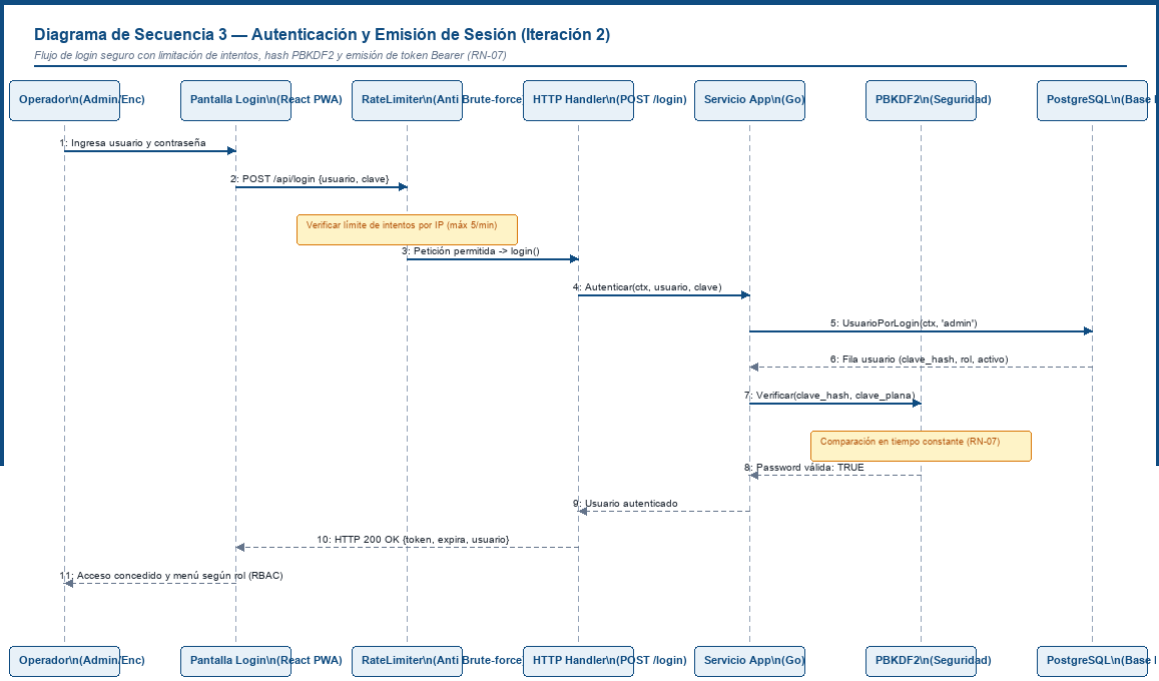



*Figura 8. Diagrama de Secuencia 3 — Autenticación y Emisión de Sesión (Iteración 2: rate limiting, hash PBKDF2 con sal y emisión de token Bearer RN-07)*

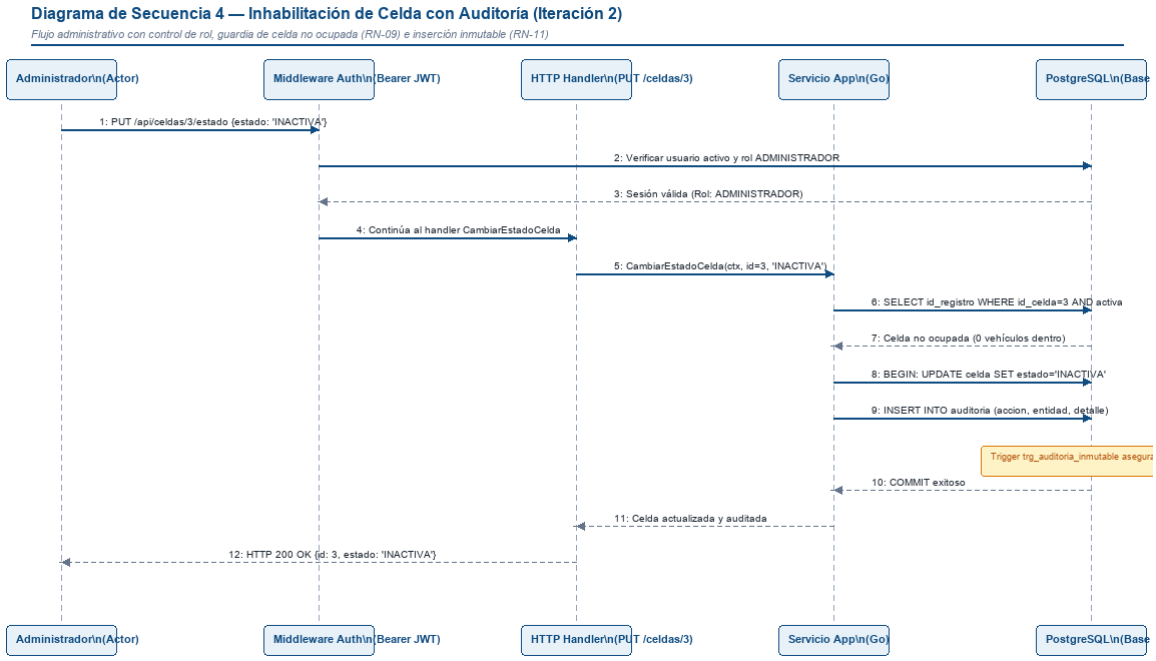



*Figura 9. Diagrama de Secuencia 4 — Inhabilitación de Celda con Auditoría (Iteración 2: validación de rol, guardia de celda ocupada RN-09 y persistencia inmutable RN-11)*

El diagrama de componentes describe la organización modular del software y las dependencias entre subsistemas. La solución implementa una **Arquitectura Hexagonal (Puertos y Adaptadores)** estricta, lo que garantiza que las reglas de negocio permanezcan completamente desacopladas de los protocolos de transporte y de la base de datos.

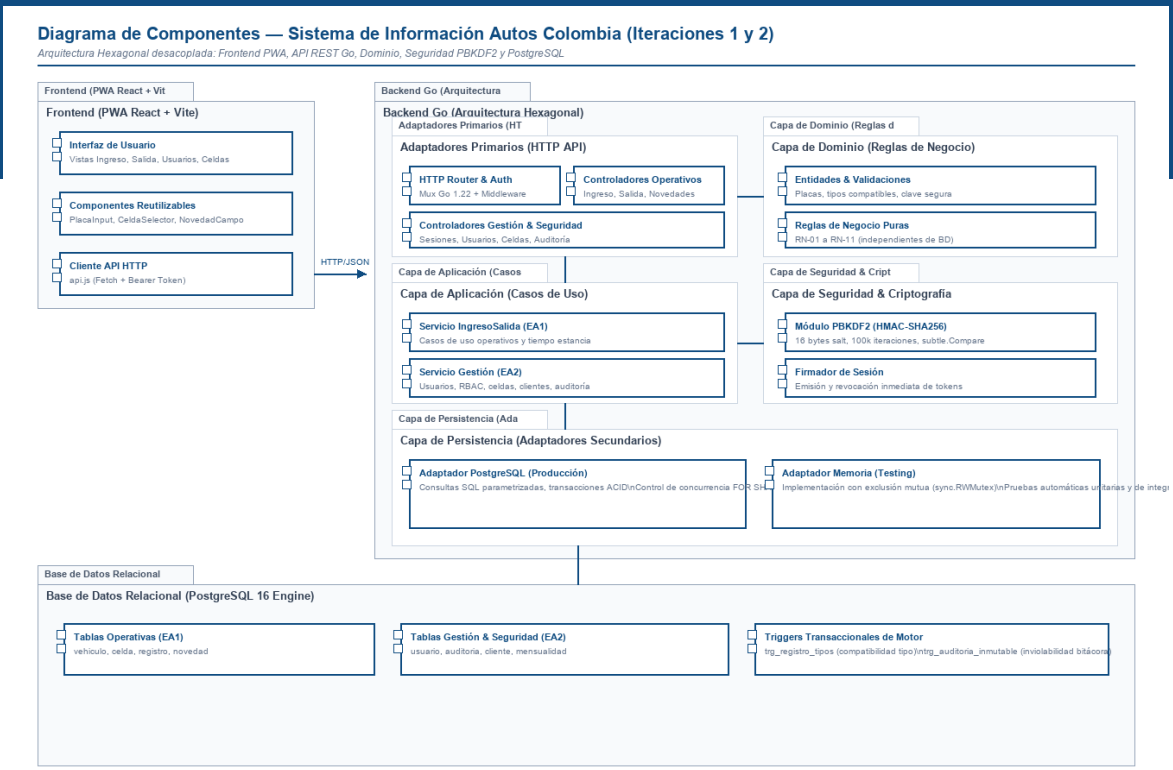



*Figura 10. Diagrama de Componentes del Sistema Autos Colombia (Iteraciones 1 y 2: Frontend PWA, Backend Go Hexagonal y PostgreSQL 16)*

## Descripción de Módulos y Capas

• **Frontend (PWA React + Vite):** Capa de presentación para dispositivos móviles y de escritorio. Contiene vistas modulares y el cliente HTTP (**api.js**) que administra las cabeceras de autorización Bearer.
• **Adaptadores Primarios (HTTP API):** Router nativo Go 1.22 con validación de métodos (405) y middleware de autenticación, control de tasa (anti fuerza bruta) y cabeceras de seguridad.
• **Capa de Aplicación:** Casos de uso (**ServicioIngresoSalida** y **ServicioGestion**) que orquestan el negocio sin conocer la base de datos concreta.
• **Dominio:** Entidades puras y validaciones invariantes (placas colombianas, cálculo de vigencia UTC-5, política de contraseñas, validación de códigos de celda).
• **Seguridad Criptográfica:** Hasheador PBKDF2 (HMAC-SHA256, 16 bytes de sal aleatoria, 100.000 iteraciones) y módulo de firma de sesiones.
• **Adaptadores Secundarios (Persistencia):** Repositorio PostgreSQL transaccional y Repositorio en Memoria para pruebas unitarias de alta velocidad.
• **Motor de Base de Datos:** PostgreSQL 16 con disparadores **trg_registro_tipos** (impide asociar carro en celda

moto a nivel físico) y **trg_auditoria_inmutable** (impide alterar la bitácora).

El aseguramiento de la calidad del software se fundamenta en pruebas automatizadas y verificaciones de integración. A continuación se presenta la matriz formal del **Plan de Pruebas Resumido** de la Iteración 2, cubriendo los escenarios de éxito, condiciones de borde y rechazos de negocio.

| Id | Módulo | Caso de Prueba | Precondiciones | Datos de Entrada | Resultado Esperado | Resultado Obtenido | Estado |
|---|---|---|---|---|---|---|---|
| CP-01 | Seguridad | Autenticación válida de usuario y emisión de sesión | Usuario registrado y activo en BD. | usuario: 'admin', clave: 'ClaveSegura 123' | HTTP 200 OK con token Bearer firmado y expiración. | Token emitido; reinicio de intentos. | Aprobado |
| CP-02 | Seguridad | Rechazo por credenciales erróneas | Usuario registrado o inexistente. | usuario: 'admin', clave: 'Invalida' | HTTP 401 Unauthorized ('usuario o contraseña incorrectos'). | HTTP 401; sin filtrar motivo. | Aprobado |
| CP-03 | Seguridad | Bloqueo por fuerza bruta tras 5 fallos | IP sin bloqueo previo. | 6 intentos fallidos consecutivos en < 60s | HTTP 429 Too Many Requests con cabecera Retry-After: 60. | HTTP 429 en el 6to intento; IP bloqueada. | Aprobado |
| CP-04 | Usuarios | Registro exitoso de nuevo usuario por Admin | Admin autenticado con token Bearer. | usuario: 'oper1', nombre: 'Carlos Ruiz', rol: 'ENCARGADO' | HTTP 201 Created. Usuario activo; auditoría generada. | Usuario persistido; hash PBKDF2 guardado. | Aprobado |
| CP-05 | Usuarios | Rechazo por nombre de usuario duplicado | Existe usuario con login 'admin'. | usuario: 'admin', rol: 'ADMINISTRADOR' | HTTP 409 Conflict ('ya existe un usuario con ese nombre'). | HTTP 409; unicidad garantizada. | Aprobado |
| CP-06 | Usuarios | Rechazo por contraseña débil o inválida | Admin autenticado. | clave: 'corta1' (< 10 caracteres) | HTTP 400 Bad Request explicando directivas de clave. | HTTP 400 con detalle de la política. | Aprobado |
| CP-07 | Usuarios | Protección del último administrador activo | Solo 1 administrador activo. | Admin intenta inactivar su cuenta o degradar rol. | HTTP 409 Conflict ('debe quedar al menos un admin activo'). | Error atómico ErrUltimoAdministrador. | Aprobado |
| CP-08 | Usuarios | Cambio personal de contraseña con clave actual | Usuario autenticado. | clave_actual: 'Vieja123', clave_nueva: 'Nueva2026' | HTTP 200 OK. Contraseña actualizada y auditada. | Hash actualizado; sesión renovada. | Aprobado |
| CP-09 | Celdas | Registro exitoso de nueva celda | Admin autenticado. | codigo: 'A-05', tipo: 'CARRO' | HTTP 201 Created. Celda en estado DISPONIBLE. | Celda creada en mayúsculas. | Aprobado |
| CP-10 | Celdas | Rechazo por código de celda duplicado | Existe celda con código 'A-01'. | codigo: 'a-01', tipo: 'CARRO' | HTTP 409 Conflict ('ya existe una celda con ese código'). | Código normalizado a mayúsculas y rechazado. | Aprobado |

| Id | Módulo | Caso de Prueba | Precondiciones | Datos de Entrada | Resultado Esperado | Resultado Obtenido | Estado |
|---|---|---|---|---|---|---|---|
| CP-11 | Celdas | Inhabilitación de celda por mantenimiento | Celda DISPONIBLE sin vehículos. | estado: 'INACTIVA' | HTTP 200 OK. Celda pasa a INACTIVA y se excluye de ingresos. | Celda inhabilitada y auditada. | Aprobado |
| CP-12 | Celdas | Bloqueo de inhabilitación en celda ocupada | Celda con vehículo en estancia. | id_celda: 2 (ocupada), estado: 'INACTIVA' | HTTP 409 Conflict ('la celda está ocupada; registre salida'). | Rechazo inmediato; celda protegida. | Aprobado |
| CP-13 | Clientes | Rechazo de mensualidad solapada | Cliente con mensualidad del 01/11 al 30/11. | periodo: 2026-11-15 al 2026-12-15 | HTTP 409 Conflict ('la mensualidad se solapa'). | Validación de fechas ejecutada. | Aprobado |
| CP-14 | Auditoría | Inmutabilidad de la tabla de auditoría | Filas existentes en tabla auditoria. | Sentencia SQL UPDATE o DELETE directa en motor. | Trigger trg_auditoria_inmutable aborta la transacción. | Excepción lanzada; registros intactos. | Aprobado |

## Evidencia de Ejecución Automatizada en el Backend

La totalidad de los casos de prueba de dominio, aplicación, seguridad y adaptadores HTTP y PostgreSQL se ejecutan mediante el comando **go test -race ./...**, obteniendo un resultado de **0 fallos y 100 % de aprobación** en todas las suites.

El código fuente completo del sistema, con la totalidad de los módulos de la Iteración 1 y la Iteración 2, se encuentra versionado bajo Git y alojado en GitHub en el siguiente enlace público:

**URL Oficial del Repositorio:**

En el commit **c928179** se incorporaron más de 4.100 líneas de código fuente en Go y sentencias DDL en PostgreSQL, distribuidas en los siguientes componentes:

| Paquete / Archivo | Propósito en la Iteración 2 | Líneas Agregadas |
|---|---|---|
| backend/internal/domain/gestion.go | Entidades Usuario, Rol, ClienteDetalle, Auditoria y reglas de validación (clave robusta, login normalizado, celdas gestionables). | +135 |
| backend/internal/application/gestion.go | Casos de uso de autenticación, control RBAC, gestión de usuarios (guardia de último admin), celdas (guardia de ocupación) y auditoría. | +421 |
| backend/internal/adapters/seguridad/pbkdf2.go | Adaptador criptográfico PBKDF2 con sal aleatoria de 16 bytes y comparación en tiempo constante (subtle.ConstantTimeCompare). | +91 |
| backend/internal/adapters/httpapi/gestion.go | Endpoints REST para /api/login, /api/sesion, /api/usuarios, /api/celdas, /api/clientes y /api/auditoria. | +311 |
| backend/internal/adapters/httpapi/sesion.go | Emisión y verificación de tokens de sesión con revocación inmediata al inactivar usuario. | +66 |
| backend/internal/adapters/postgres/gestion.go | Consultas SQL parametrizadas, transacciones ACID y bloqueos pesimistas para PostgreSQL. | +376 |
| backend/db/schema.sql | Tablas usuario y auditoria; triggers trg_auditoria_inmutable y trg_registro_tipos. | +58 |
| Suites de Pruebas (*_test.go) | Pruebas unitarias y de integración para gestión, seguridad, handlers HTTP y PostgreSQL real. | +2.600+ |

## 10. Matriz de Trazabilidad entre Representaciones

La trazabilidad garantiza que cada requerimiento funcional esté respaldado por una historia de usuario, un caso de uso, componentes de software, tablas en la base de datos, endpoints de API y casos de prueba verificados.

| Requisito | Historia | Caso de Uso | Componentes / Clases | Tablas en BD | Endpoint API | Prueba |
|---|---|---|---|---|---|---|
| RF-08, RF-09 | HU-07 | CU-06 | Router, MiddlewareAuth, SvcGestion, PBKDF2 | usuario | POST /api/login | CP-01, CP-02, CP-03 |
| RF-10, RF-11 | HU-06 | CU-08 | CtrlGestion, SvcGestion, Dominio, RepoPG | usuario, auditoria | GET, POST, PATCH /api/usuarios | CP-04, CP-05, CP-07 |
| RF-12, RF-13 | HU-07 | CU-07 | CtrlGestion, SvcGestion, PBKDF2 | usuario, auditoria | POST /api/sesion/clave | CP-06, CP-08 |
| RF-15, RF-16, RF-17 | HU-08 | CU-09 | CtrlGestion, SvcGestion, RepoPG | celda, auditoria | GET, POST, PUT /api/celdas | CP-09, CP-10, CP-11, CP-12 |
| RF-18, RF-19 | HU-09 | CU-10 | CtrlGestion, SvcGestion, Dominio | cliente, vehiculo, mensualidad | GET, POST /api/clientes | CP-13 |

| Requisito | Historia | Caso de Uso | Componentes / Clases | Tablas en BD | Endpoint API | Prueba |
|---|---|---|---|---|---|---|
| RF-20, RF-21 | HU-10 | CU-11 | CtrlGestion, SvcGestion, Triggers | auditoria (trigger inmutable) | GET /api/auditoria | CP-14 |

1. **Evolución hacia la madurez arquitectónica:** La Iteración 2 permitió transformar un sistema monousuario básico en una plataforma empresarial robusta dotada de gobernabilidad, seguridad y trazabilidad. La implementación del control de acceso basado en roles (RBAC) garantiza que las acciones críticas de configuración del patio y administración de identidades queden reservadas al Administrador, protegiendo la operación.

2. **Seguridad integral y no repudio:** La combinación de contraseñas protegidas mediante PBKDF2 con sal individual, comparaciones en tiempo constante, protección perimetral contra fuerza bruta (Rate Limiting) y una bitácora de auditoría inmutable mediante triggers transaccionales en PostgreSQL provee un estándar de seguridad de nivel de producción que cumple plenamente con los postulados de la ingeniería de software moderna.

3. **Integridad del modelo de negocio en persistencia:** Reglas de negocio críticas como la imposibilidad de inactivar al último administrador activo, la prohibición de inhabilitar celdas ocupadas y la compatibilidad estricta de tipos celda-vehículo fueron blindadas tanto en la lógica de aplicación como a nivel de restricciones de base de datos, eliminando la posibilidad de estados corruptos ante accesos concurrentes.

4. **Coherencia y trazabilidad absoluta:** Se demostró una alineación perfecta entre las necesidades de negocio, los requerimientos formales, los modelos UML en StarUML (casos de uso, secuencias y componentes), las interfaces de usuario (mockups en Figma) y las pruebas automatizadas (100 % aprobadas), garantizando un ciclo de desarrollo ágil y riguroso.